AI ASSISTANT CODING ASSIGNMENT-9.3

Name: D. Divya Sri

HT. No: 2303A51323

Batch: 19

Task 1: Basic Docstring Generation

Code: Manual code with manual Google-style Docstring

```python
def sum_even_odd(numbers):
    """
    Calculates the sum of even and odd numbers in a given list.
    Args:
        numbers (list[int]): A list of integers.
    Returns:
        tuple[int, int]: A tuple containing:
            - Sum of even numbers
            - Sum of odd numbers
    """
    even_sum = 0
    odd_sum = 0
    for num in numbers:
        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num
    return even_sum, odd_sum
# Example usage
numbers = [1, 2, 3, 4, 5, 6]
even_sum, odd_sum = sum_even_odd(numbers)
print(f"Sum of even numbers: {even_sum}")
print(f"Sum of odd numbers: {odd_sum}")
```

Prompt: Generate a Google-style function-level docstring for a Python function that takes a list of integers and returns a tuple containing the sum of even numbers and the sum of odd numbers.

Include:

Short description

Args section

Returns section

Proper data types

AI-Generated Docstring

"""
Compute the total sum of even and odd integers from a list.
Args:
  numbers (List[int]): A list of integer values.
Returns:
  Tuple[int, int]: A tuple where the first element is the
  sum of even integers and the second element is the sum
  of odd integers.
"""

Analysis: The manual docstring was concise and directly matched the function requirements. The AI-generated docstring was slightly more detailed and formal in structure. Both were correct and complete, but AI added extra explanation. Manual writing gives better control, while AI improves speed.

Result:

```
Sum of even numbers: 12
Sum of odd numbers: 9
```

Observation:

The AI-generated docstring followed the Google style format correctly and provided clear parameter and return descriptions. However, it was slightly more verbose than necessary and added formal type hints that were not strictly required. The manual docstring was more concise and directly aligned with the function's purpose. While AI improves speed and consistency, human refinement ensures clarity and precision.

Task 2: Automatic Inline Comments

Code: Python Class with Manual Inline Comments

```python
#Task 2
class sru_student:
    # Constructor to initialize student attributes
    def __init__(self, name, roll_no, hostel_status):
        self.name = name  # Store student name
        self.roll_no = roll_no  # Store roll number
        self.hostel_status = hostel_status  # Store hostel status (Yes/No)

    # Method to update fee details
    def fee_update(self, amount):
        print(f"Fee updated by {amount}")  # Display updated fee amount

    # Method to display student details
    def display_details(self):
        print("Name:", self.name)  # Print student name
        print("Roll No:", self.roll_no)  # Print roll number
        print("Hostel Status:", self.hostel_status)  # Print hostel status
# Create an object of sru_student class
student1 = sru_student("John Doe", "12345", "Yes")
student1.display_details()  # Display student details
student1.fee_update(5000)  # Update fee with an amount of 5000
```

Prompt: Add meaningful inline comments to the following Python class.Explain logical blocks clearly. Do not modify the code structure.

Code: AI-Generated Inline Comments

```python
#prompt:Add meaningful inline comments to the following Python class.Explain logical blocks clearly.Do not modify the code structure.
class sru_student:
    # Initialize the student object with basic details
    def __init__(self, name, roll_no, hostel_status):
        self.name = name  # Assign the name to the instance variable
        self.roll_no = roll_no  # Assign the roll number
        self.hostel_status = hostel_status  # Assign hostel status

    # Update student fee information
    def fee_update(self, amount):
        print(f"Fee updated by {amount}")  # Output the updated fee message

    # Display all stored student information
    def display_details(self):
        print("Name:", self.name)  # Show student name
        print("Roll No:", self.roll_no)  # Show roll number
        print("Hostel Status:", self.hostel_status)  # Show hostel info
# Create an instance of the sru_student class with sample data
student1 = sru_student("John Doe", "12345", "Yes")
student1.display_details()  # Call method to display student details
student1.fee_update(5000)  # Call method to update fee with a specific amount
```

Result:

```
Name: John Doe
Roll No: 12345
Hostel Status: Yes
Fee updated by 5000
```

Analysis: Manual inline comments were simple and focused on important logic. AI-generated comments were more detailed but sometimes redundant. AI helps improve readability, but manual refinement is needed to avoid unnecessary explanations.

Observation:

The AI-generated inline comments explained nearly every line of code, improving readability for beginners but sometimes creating redundancy. The manual comments were shorter and focused only on important logical sections. Although AI enhances understanding and documentation speed, it may over-explain simple statements. Human review is necessary to maintain clean and professional commenting standards.

Task3: Module-Level and Function-Level Documentation

Code: Calculator Script with Manual NumPy Docstrings

```
"""
calculator_module.py
Provides basic arithmetic operations.
"""
def add(a, b):
    """Add two numbers."""
    result = a + b
    print("Addition:", result)
    return result
def subtract(a, b):
    """Subtract two numbers."""
    result = a - b
    print("Subtraction:", result)
    return result
def multiply(a, b):
    """Multiply two numbers."""
    result = a * b
    print("Multiplication:", result)
    return result
def divide(a, b):
    """Divide two numbers."""
    if b == 0:
        raise ValueError("Cannot divide by zero")
    result = a / b
    print("Division:", result)
    return result
add(10, 5)
subtract(10, 5)
multiply(10, 5)
divide(10, 5)
```

Prompt:

#Generate a NumPy-style module-level docstring for a calculator module
#that includes add, subtract, multiply, and divide functions.
#Include overview and usage.
#Generate NumPy-style docstrings for add, subtract, multiply, divide functions.
#Include Parameters, Returns, and Raises (for divide).

Code: AI Generated

```
calculator_module.py
====================
A simple calculator module that provides basic arithmetic operations:
addition, subtraction, multiplication, and division.
Overview
--------
This module contains four functions:
- add(a, b)
- subtract(a, b)
- multiply(a, b)
- divide(a, b)

Each function performs a basic arithmetic operation on two numeric inputs.
Usage
-----
>>> import calculator_module as calc
>>> calc.add(10, 5)
15
>>> calc.divide(10, 2)
5.0
"""
def add(a, b):
    """
    Add two numbers.
    Parameters
    ----------
    a : int or float
        First number.
    b : int or float
        Second number.
    Returns
    -------
    int or float
        Sum of a and b.
    """
    return a + b
def subtract(a, b):
    """
    Subtract two numbers.
    Parameters
```

```
def subtract(a, b):
    int or float
        Difference of a and b.
    """
    return a - b
def multiply(a, b):
    """
    Multiply two numbers.
    Parameters
    ----------
    a : int or float
        First number.
    b : int or float
        Second number.
    Returns
    -------
    int or float
        Product of a and b.
    """
    return a * b
def divide(a, b):
    """
    Divide two numbers.
    Parameters
    ----------
    a : int or float
        Numerator.
    b : int or float
        Denominator.
    Returns
    -------
    float
        Result of division.
    Raises
    ------
    ValueError
        If b is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Result:

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```

Analysis: The manual NumPy docstrings were accurate and structured properly. The AI-generated documentation was more descriptive and detailed. Both were correct, but AI tends to be more verbose. Human review ensures clarity and precision.

Observation:

The AI-generated module and function-level documentation correctly followed the NumPy format and included detailed parameter, return, and raise sections. However, the descriptions were slightly more elaborate than required. The manual documentation was accurate and structured but more concise. This task shows that AI is highly effective for generating structured documentation, but manual customization ensures better alignment with project-specific needs.
odules, which may be necessary in certain execution environments.